

Operating system principles 5

Paul Blondel

CNAM

September 1st, 2018

'Some people have told me they don't think a fat penguin really embodies the grace of Linux, which just tells me they have never seen an angry penguin charging at them in excess of 100mph. They'd be a lot more careful about what they say if they had.'

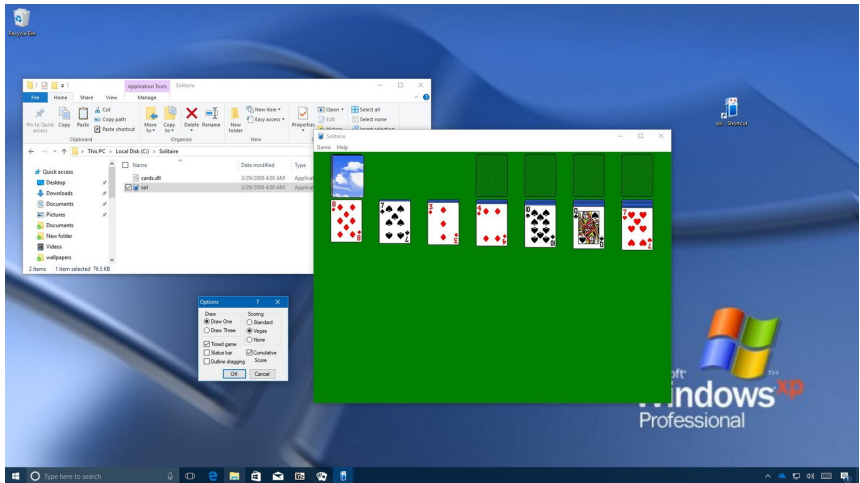
Linus Torvalds



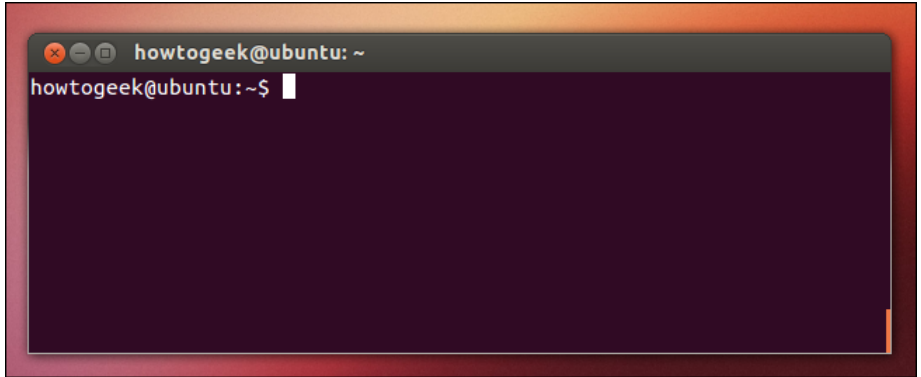
- 1 The shell
- 2 The command line
- 3 The Shell prompt
- 4 Typing into the Shell
- 5 Navigating the File System
- 6 Permissions
- 7 Compression, archiving and wildcards
- 8 Shell redirection & Pipeline

- 9 Editing text files
- 10 File transfers
- 11 Additional commands
- 12 Environment Variables
- 13 More commands and tips...

What you have used so far:

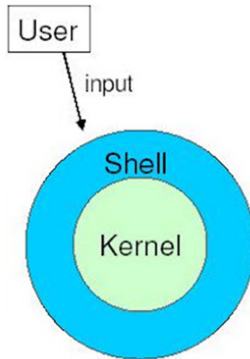


What you will always use after this course:



The Shell:

- Shell is an application that reads command lines from the keyboard and passes them to the Linux operating system to be executed
- When you login to a remote Linux system, using a tool like ssh, you will automatically be connected to a shell
- Your computing session is kept separate from other user's computing sessions, because they are “enclosed” in a “shell”
- On your desktop, laptop, or tablet, you may have to find and execute a terminal emulator application to bring up a shell in a window



Definition

The *Shell* is basically a bridge between kernel and the user. Using *command lines* users can type commands which are conveyed to the kernel and will be executed.

A basic way of interacting with a Linux system:

- Execute commands
- Create files and directories
- Edit file content
- Access the web
- Copy files to and from other hosts
- Run parallel jobs on distant servers
- etc ...
- ... basically: do things you cannot do as easily with a Graphical User Interface (GUI)

Why learning the command line?

- Linux was designed for the command line
- You can create new Linux commands using the command line, without programming
- Many systems provide only the command line, or poorly support a GUI interface
 - ▶ HPC systems
 - ▶ Spark/Hadoop for Big Data
 - ▶ etc...
- Many things can be accomplished only through the command line
 - ▶ Much systems administration & troubleshooting

Why learning the command line?

- Linux was designed for the command line
- You can create new Linux commands using the command line, without programming
- Many systems provide only the command line, or poorly support a GUI interface
 - ▶ HPC systems
 - ▶ Spark/Hadoop for Big Data
 - ▶ etc...
- Many things can be accomplished only through the command line
 - ▶ Much systems administration & troubleshooting
- Or you just want to look "cool"



To access a Shell you need to use a Terminal Emulator:

- On Linux and Mac OS X:
 - ▶ Just start a terminal (Xterm, Eterm, etc)
 - ▶ Use ssh command (if you want to connect to another machine's Shell)
- On Windows:
 - ▶ You can use Cygwin to launch Linux commands on your system
 - ▶ Use putty (or mobaxterm) to connect to another machine's Shell

The “user@machine:~\$” is the shell prompt

- This means the shell is waiting for you to type something
- Format can vary, usually ends with “\$” , “%” or “#”
 - ▶ If \$ or %, you have a normal shell
 - ★ This shell has your privileges (rights)

The “user@machine:~\$” is the shell prompt

- This means the shell is waiting for you to type something
- Format can vary, usually ends with “\$” , “%” or “#”
 - ▶ If \$ or %, you have a normal shell
 - ★ This shell has your privileges (rights)
 - ▶ If #, you have a so-called “root shell”
 - ★ This shell has administrator privileges
 - ★ You can do a great deal of irreversible damage



Basic input line editing commands:

- Backspace erases previous character
- Left and right arrow move insertion point on the line
- Control-c interrupts whatever command you started and returns you to the shell prompt (usually)
- Control-u erases the line from the beginning to the cursor
- Control-k erases the line from the cursor to the end
- Enter executes the line you typed
- Up and down arrow will access your command history
- Type "exit" and press Enter without the quotes to exit the shell
- Click the red "close" icon at the top of the Terminal window to close it (on a Mac)

Later, as a practical exercise, try these commands and see what happen:

~\$date

~\$ id

~\$ ps

~\$ df -kh

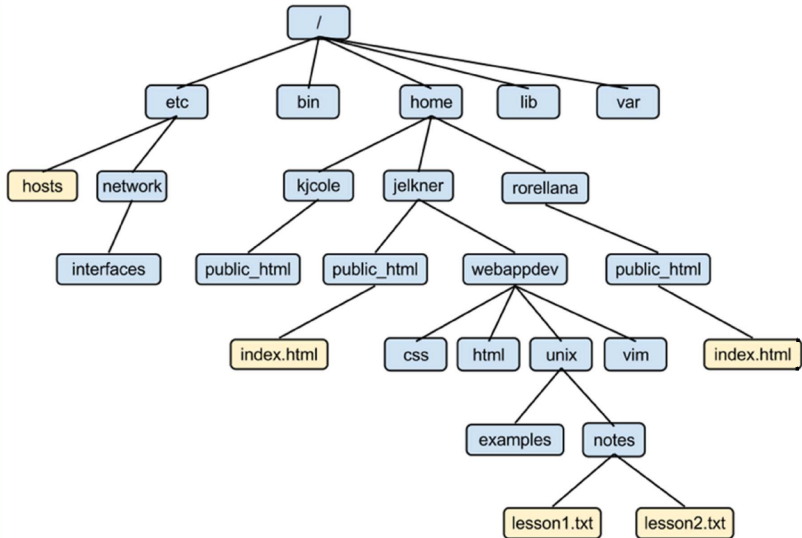
~\$ who

~\$ top # type Control-c or q to exit

~\$ history

Recall:

- Files are stored in a directory
- Directories may contain other directories as well as files
- A hierarchy of directories is called a directory tree
- A directory tree (a connected graph with no cycles) has a single, topmost root directory
- A directory tree, rooted at the system root directory “/”, is called a file system



- A file is accessed using its path name
- Absolute path name
 - ▶ /dir1/dir2/.../dirN/filename
 - ▶ /usr/X11R6/bin
- Relative path name
 - ▶ current-working-directory/filename
- Every shell maintains a notion of a current working directory
 - ▶ Initialized at login to your home directory
 - ▶ Changed via cd command
- Two special directories:
 - ▶ . refers to the current directory
 - ▶ .. refers to the current directory's parent directory
- Many ways to get "home"
 - ▶ ~ refers to your home directory
 - ▶ \$HOME is a synonym for ~
 - ▶ ~username refers to a user's home directory

Some fundamental commands:

- ~\$ `file file` # what kind of file is file?
- ~\$ `cat file` # display contents of text file
- ~\$ `less file` # paginate text file
- ~\$ `man command` # get info about command

Exercise for later

Figure out how to make the `date` command display the date in Coordinated Universal Time (UTC)

Some fundamental commands:

- ~\$ `pwd` # print working directory
- ~\$ `cd dir` # make dir the current working directory
- ~\$ `cd` # cd to your home dir
- ~\$ `cd cja` # cd to cja's home dir
- ~\$ `mkdir dir` # create directory dir
- ~\$ `rmdir dir` # remove (empty) directory dir
- ~\$ `rm -fR dir` # remove directory dir (empty or not)
- ~\$ `tree` # display dir tree

Later, as a practical exercise, try these commands and see what happen:

- ~\$ `cd` # make your home directory the current working directory
- ~\$ `pwd` # print working directory
- ~\$ `mkdir foo` # create directory foo
- ~\$ `cd foo` # cd to the foo directory
- ~\$ `mkdir bar` # create directory bar
- ~\$ `cd ..` # go up one level in the directory tree
- ~\$ `tree foo` # display foo's directory tree.

The `ls` command (list information about files)

~\$ `ls` # list contents of cur dir

~\$ `ls dir` # list contents of dir

~\$ `ls -l` # list details of files in current dir

~\$ `ls -t` # list newest files first

~\$ `ls -R dir` # list all files in tree dir

~\$ `ls -lt dir` # options can be combined

~\$ `ls -hl dir` # list all files in human readable format

These commands manipulate files:

- ~\$ mv big large # rename file big to large
- ~\$ cp big large # copy file big to large
- ~\$ cp -r dir1 dir2 # copy dir tree dir1 to dir2
- ~\$ cp f1 f2 dir # copy file1 and file2 to directory dir
- ~\$ mkdir dir # create empty directory dir

These commands manipulate files:

- ~\$ mv big large # rename file big to large
- ~\$ cp big large # copy file big to large
- ~\$ cp -r dir1 dir2 # copy dir tree dir1 to dir2
- ~\$ cp f1 f2 dir # copy file1 and file2 to directory dir
- ~\$ mkdir dir # create empty directory dir
- ~\$ rmdir dir # remove empty directory dir
- ~\$ rm file # remove file file
- ~\$ rm -r dir # remove directory tree dir



Later, as a practical exercise, let's do this:

Create a directory named `tutorial` in your home directory. In that directory, create a directory named `sample` and a directory named `test` . Create a file named `msg` in directory `test` that contains a copy of the file `/etc/os-release`.

Extra credit

Make the last-modified time of your copy identical to that of `/etc/os-release`. Hint: look at the options of the `copy` command

Recall:

- We have three permission types:
 - ▶ Files: Read, Write and Execute
 - ▶ Equivalent for directories: List, Modify and Search
- We have three user classes:
 - ▶ User (File Owner), Group, All (Other)
 - ▶ Type: `man chmod` (for more help)

```
shum@sol:~$ ls -l
total 20
drwx----- 2 shum      staff      4096 Jan 16 22:04 Mail
drwx----- 3 shum      staff      4096 Jan 16 14:15 csc128
drwxr-xr-x  2 shum      staff      4096 Jan 13 16:42 public
drwxr-xr-x  2 shum      staff      4096 Jan 16 14:07 public_html
-rw-r--r--  1 shum      staff       628 Jan 15 20:04 verse
```

Diagram illustrating the components of the `ls -l` command output:

- file type**: Points to the first character of the permissions (e.g., `d` for directory, `-` for regular file).
- number of hard links**: Points to the number before the owner name (e.g., `2`).
- user (owner) name**: Points to the owner name (e.g., `shum`).
- group name**: Points to the group name (e.g., `staff`).
- size**: Points to the size in bytes (e.g., `4096`).
- date/time last modified**: Points to the date and time (e.g., `Jan 16 22:04`).
- filename**: Points to the filename (e.g., `Mail`).
- permissions**: A box highlights the permissions `drwxr-xr-x` with arrows pointing to:
 - other (everyone) permissions** (red): Points to the last three characters (`r-xr-x`).
 - group permissions** (cyan): Points to the middle three characters (`wxr-xr-x`).
 - user permissions** (green): Points to the first three characters (`drwxr-xr-x`).
- executable**: Points to the `x` in `rwx`.
- writeable**: Points to the `w` in `rwx`.
- readable**: Points to the `r` in `rwx`.

Other examples:

`-rw----- cja lsait 40 Oct 1 12:03 foo.bz2`

file read and write rights for the owner, no access for anyone else

`-rw-r--r-- cja lsait 40 Oct 1 12:03 foo.bz2`

file read and write rights for the owner, read for members of the lsait group and no access for others (we typed: `chmod u=rw,g=r,o= foo.bz2`)

`drwxr-xr-x cja lsait 4096 Oct 1 12:15 bar`

list, modify, and search for the owner, list and search for group, and execute only for others

These commands compress and archive files:

```
~$ gzip foo # compress foo to foo.gz
~$ gunzip foo # uncompress foo.gz to foo
~$ bzip2 foo # better compress foo to foo.bz2
~$ bunzip2 foo # uncompress foo.bz2 to foo
~$ tar -cf foo.tar bar # archive subtree bar in file foo.tar
~$ tar -xf foo.tar # restore archive from file foo.tar
~$ tar -tf foo.tar # list files in archive file foo.tar
~$ tar -zcf foo.tgz bar # archive and compress
~$ tar -jcf foo.tjz bar # archive and compress better
```

Exercise for later

Archive and compress files in a directory to a file named example.tgz

The Shell accepts wildcarded arguments (also called "shell globbing")

Wildcards:

- `?` Matches a single character
- `*` Matches zero or more characters
- `[chars]` Matches any of the chars
- `[c1-c2]` Matches chars 'c1' through 'c2'
- `[~chars]` Matches any but the chars

`~$ ls foo.?` # match files named foo.x, where x is any character

`~$ echo .[cs]` # echo files that end in .c or .s

`~$ mv [o-z]* save` # move files starting with o through z to directory save

`~$ echo [~A-Z]?` # ???

A Linux command can have its inputs and outputs redirected:

```
~$ ls >myfiles # put list of files in current directory into file myfiles
~$ ls »filelist # add list of files in current directory to end of file filelist
~$ sort <grocery.list # sort lines from file grocery.list
~$ sort «EOF
whiskey
bravo
tango
EOF # sort lines entered at keyboard
~$ wc -l </etc/os-release >~/mycounts # count number of lines from
file /etc/os-release and put result in file mycounts in my home directory
```


More useful Linux tool commands used for redirection and pipelines:

- ~\$ `grep string` # show lines of input containing string
- ~\$ `tail` # show last few lines of input
- ~\$ `head` # show first few lines of input
- ~\$ `sort` # sort the input
- ~\$ `du -sh` # report the size of the current directory
- ~\$ `du -sh dir` # report the size of directory dir
- ~\$ `who` # gives a list of the users currently logged in
- ~\$ `cut -cxx -yy` # keep the output from a command starting at the xx character in the line and ending at the yy character

A Linux command can have its output connected to the input of another Linux command:

~\$ `ls | wc -l` # count files in current directory

~\$ `last | grep reboot` # when did we reboot?

Exercises for later

- How many users are currently logged in?
- How many unique user IDs are currently logged in?

Simple editor:

- nano
 - ▶ Simple to learn if you want to get started quickly
 - ▶ No mouse support. Arrow keys for navigation

Supported editors:

- vi, vim or emacs
 - ▶ Powerful but more complex
 - ▶ If you have time and inclination to become proficient

Warning

Watch out for source code or data files written on Windows systems

Use these tools to analyze and convert source files to Linux format:

- file
- dos2unix
- unix2dos

Eventually, you will need to move/copy files to and from your computer and a server/workstation/cluster. You can do this using the secure copy command (scp) in a terminal window.

To transfer files (foobar.txt) FROM your local host TO a remote host use:

```
~$ scp foobar.txt username@remotehost.edu:/some/remote/directory
```

To transfer files (foobar.txt) FROM a remote host TO your local host use:

```
~$ scp username@remotehost.edu:foobar.txt /some/local/directory
```

To copy a directory, repeat as above adding the -r flag. (~\$ scp -r ...)

Graphical, drag-and-drop scp programs are available for Windows and Mac platforms. (WinSCP – Windows, Cyberduck – Mac)

Modern operating systems are usually multitasking, meaning that they create the illusion of doing more than one thing at once by rapidly switching from one executing program to another. The Linux kernel manages this through the use of processes. Processes are how Linux organizes the different programs waiting for their turn at the CPU.

- On Linux, every program runs in a process
- You can examine these processes

Commands on processes:

~\$ man ps

~\$ ps

~\$ ps ax

~\$ top

Modern operating systems are usually multitasking, meaning that they create the illusion of doing more than one thing at once by rapidly switching from one executing program to another. The Linux kernel manages this through the use of processes. Processes are how Linux organizes the different programs waiting for their turn at the CPU.

- On Linux, every program runs in a process
- You can examine these processes

Commands on processes:

~\$ man ps

~\$ ps

~\$ ps ax

~\$ top

~\$ man kill

~\$ kill PID



Additional interesting commands:

- ~\$ `quota -Q -s $USER` # show disk quota for \$USER
- ~\$ `grep "sometext" somefile` # find & print sometext if found in somefile
- ~\$ `history` # displays last n commands entered
- ~\$ `clear` # clears the screen
- ~\$ `diff -w file1 file2` # compare file1 with file2 ignoring all white space
- ~\$ `which command` # prints the full path to command
- ~\$ `acommand | tee filename` # takes the results from acommand and prints them to the screen and to the file filename (Use tee -a to append to filename)
- ~\$ `source filename` # reads and executes the commands contained in filename

An environment variable is a named object that contains data used by one or more applications. In simple terms, it is a variable with a name and a value.

The convention in Linux is for the environment variable to be all uppercase letters.

To use an environment variable, prefix the variable name with \$. You can see the value of an environment variable by typing:

```
~$ echo $VARIABLENAME
```

You can see all the environment variables defined for your session by typing:

```
~$ env
```

You can set an environment variable by typing:

```
~$ export VARIABLENAME = value
```

Common environment variables:

USER # \$USER -> user login name

PATH # \$PATH -> a list of directories to look into for programs and files

HISTSIZE # \$HISTSIZE -> number of commands to keep in history

HOSTNAME # \$HOSTNAME -> fully enumerated host name

HOME # \$HOME -> home directory of the user

TERM # \$TERM -> terminal type

SHELL # \$SHELL -> shell type you are using

You can find more commands and tips in the "Linux Command Line" book:

<http://www.linuxzasve.com/preuzimanje/TLCL-09.12.pdf>